



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
по курсу «Разработка параллельных и
распределенных программ»

Задача о пяти обедающих философах.

Студент: Федуков А. А.

Группа: ИУ9-52Б

Преподаватель: Царев А. С.

Москва, 2025 г.

Содержание

Постановка задачи	2
Практическая реализация	4
Код программы	4
Вывод программы	9
Вывод	11
Характеристики компьютера	11

Постановка задачи

Пять философов сидят за круглым столом и по очереди думают и едят. В центре обеденный сосуд со спагетти. Между каждыми двумя соседями лежит по одной вилке (всего 5 вилок). Для того чтобы поесть, философу требуется две вилки — левая и правая. Философ не может захватить сразу обе вилки: он сначала берёт одну (накладывая блокировку), затем — вторую; при окончании еды он освобождает обе вилки (снимает блокировки), что может происходить одновременно.

Задача — реализовать программу-имитацию следующей модели и обеспечить её корректность и отсутствие взаимоблокировок.

Требования:

1. Модель:

- Каждый философ реализуется как отдельный поток.
- В каждый момент каждый философ находится в одном из состояний: размышляет, берёт левую вилку, берёт правую вилку, ест, кладёт вилки.
- Времена размышлений и приёма пищи задаются генератором случайных чисел в заданных интервалах.

2. Синхронизация:

- Вилки — разделяемый ресурс, представленный отдельными примитивами блокировки (по одной на вилку).
- Нельзя захватывать сразу две блокировки; последовательность действий должна быть: наложить блокировку на первую вилку — взять первую вилку — наложить блокировку на вторую вилку — взять вторую вилку.
- Освобождение вилок может происходить одновременно (снятие обеих блокировок).
- Необходимо избежать ситуации взаимоблокировки, когда все философы ждут вторую вилку (например, реализовать стратегию официанта/семафора или изменение порядка взятия вилок для одного философа).

3. Ограничения и свойства:

- Одновременно могут есть не более двух философов (из-за ограниченного числа вилок).
- Соседи не могут есть одновременно.
- Программа должна корректно завершаться по истечении заданного времени симуляции: все потоки останавливаются, ресурсы освобождаются, логгер корректно выводит финальные состояния.

4. Вывод и логирование:

- В ходе работы программа должна логировать события: метка времени (мс от старта), идентификатор философа и его состояние.
- Формат вывода — табличный лог, показывающий состояние каждого философа во времени (пример: t(ms) | P00 P01 ...).

5. Сдача работы:

- Исходный код программы.
- Пример вывода работы программы (лог) при запуске симуляции.

- Краткий отчёт с описанием использованной стратегии предотвращения взаимоблокировок и кратким анализом корректности.

Параметры, которые должны быть легко настраиваемы в программе: число философов (по умолчанию 5), длительность симуляции, интервалы для времени размышления и приёма пищи, а также стратегия предотвращения взаимоблокировок (например, семафор/официант, изменение порядка захвата вилок для одного философа и т.п.).

Практическая реализация

Код программы

Листинг 1: Исходный код программы

```

1 use rand::Rng;
2 use std::{
3     fmt,
4     sync::{
5         Arc, Condvar, Mutex,
6         atomic::{AtomicBool, Ordering},
7         mpsc,
8     },
9     thread,
10    time::{Duration, Instant},
11 };
12
13 const N: usize = 5; // Число философов
14 const SECONDS: u64 = 1; // Время симуляции
15
16 #[derive(Clone, Copy, Debug)]
17 enum State {
18     Thinking,
19     TakingLeft,
20     TakingRight,
21     Eating,
22     PuttingDown,
23 }
24
25 impl fmt::Display for State {
26     fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
27         let s = match self {

```

```

28         State::Thinking => "THINK",
29         State::TakingLeft => "TAKE_L",
30         State::TakingRight => "TAKE_R",
31         State::Eating => "EAT",
32         State::PuttingDown => "PUT",
33     };
34     write!(f, "{s}")
35 }
36 }
37
38 struct Waiter {
39     permits: Mutex<usize>,
40     cv: Condvar,
41 }
42
43 impl Waiter {
44     fn new(initial: usize) -> Self {
45         Self {
46             permits: Mutex::new(initial),
47             cv: Condvar::new(),
48         }
49     }
50
51     /// Возвращает false, если надо завершаться.
52     fn acquire(&self, running: &AtomicBool) -> bool {
53         // Блокируем мьютекс
54         let mut p = self.permits.lock().unwrap();
55         while *p == 0 {
56             // Проверяем общий флаг завершения
57             if !running.load(Ordering::Relaxed) {
58                 return false;
59             }
60             // Ждем свободной вилки (с таймаутом)
61             let (pp, _) = self.cv.wait_timeout(p, Duration::from_millis(
62                 50)).unwrap();
63             p = pp;
64         }
65         // Выдаем одну вилку
66         *p -= 1;
67         true
68     }
69
70     fn release(&self) {
71         let mut p = self.permits.lock().unwrap();
72         // Освобождаем вилку
73         *p += 1;

```

```

73         // Уведомляем что вилка освобождена
74         self.cv.notify_one();
75     }
76 }
77
78 // Событие для логгера
79 #[derive(Debug)]
80 struct Event {
81     t_ms: u128,
82     id: usize,
83     state: State,
84 }
85
86 fn main() {
87     assert!(N >= 2, "N должно быть >= 2");
88
89     // Запоминаем стартовое время
90     let start_time = Instant::now();
91
92     // Общий флаг работы
93     let running = Arc::new(AtomicBool::new(true));
94
95     // Создаем вилки
96     let forks: Arc<Vec<Mutex<()>>> = Arc::new((0..N).map(|_| Mutex::new(
97         ())).collect());
98
99     // Создаем официанта, разрешаем максимум N - 1 философов одновремен
100     но
101     let waiter = Arc::new(Waiter::new(N));
102
103     // Создаем канал для логов
104     let (tx, rx) = mpsc::channel::<Event>();
105
106     // Создаем логгер как отдельный поток
107     let logger_n = N;
108     let logger = thread::spawn(move || {
109         let mut states = vec![State::Thinking; logger_n];
110
111         print!("t(ms) | ");
112         for i in 0..logger_n {
113             print!("P{i:02} ");
114         }
115         println!();
116         println!("{}", "-".repeat(7 + 1 + logger_n * 7)); // под ширину
117         "TAKE_L "
118     });

```

```

116         while let Ok(ev) = rx.recv() {
117             states[ev.id] = ev.state;
118             print!("{:>5} | ", ev.t_ms);
119             for i in 0..logger_n {
120                 print!("{:>6} ", states[i]);
121             }
122             println!();
123         }
124     });
125
126     // Запускаем философов
127     let mut thread_handles = Vec::with_capacity(N);
128     for id in 0..N {
129         // Клонировем ссылки для потоков
130         let forks = Arc::clone(&forks);
131         let waiter = Arc::clone(&waiter);
132         let running = Arc::clone(&running);
133         let tx = tx.clone();
134         let start = start_time;
135
136         let thread_handle = thread::spawn(move || {
137             let mut rng = rand::thread_rng();
138             let left = id;
139             let right = (id + 1) % N;
140
141             // Последний философ берёт сначала правую, потом левую.
142             let reverse = id == N - 1;
143
144             // Функция отправки состояния в логгер
145             let send_state = |state: State, tx: &mpsc::Sender<Event>| {
146                 let _ = tx.send(Event {
147                     t_ms: start.elapsed().as_millis(),
148                     id,
149                     state,
150                 });
151             };
152
153             while running.load(Ordering::Relaxed) {
154                 // THINK
155                 send_state(State::Thinking, &tx);
156                 thread::sleep(Duration::from_millis(rng.gen_range
(80..250)));
157
158                 // Завершаемся, если официант не дал разрешение
159                 if !waiter.acquire(&running) {
160                     break;

```

```

161         }
162
163         // Определяем взятия вилок (для последнего философа в об
ратном порядке)
164         let (first_idx, second_idx, first_state, second_state) =
if !reverse {
165             (left, right, State::TakingLeft, State::TakingRight)
166         } else {
167             (right, left, State::TakingRight, State::TakingLeft)
168         };
169
170         // Берем первую вилку
171         // Запускаем цикл с попытками взять вилку
172         send_state(first_state, &tx);
173         let first_guard = loop {
174             // Если общее завершение, то отпускаем официанта и в
ыходим
175             if !running.load(Ordering::Relaxed) {
176                 waiter.release();
177                 return;
178             }
179             // Пытаемся взять вилку
180             if let Ok(g) = forks[first_idx].try_lock() {
181                 break g;
182             }
183             // Пауза перед повторной попыткой
184             thread::sleep(Duration::from_millis(5));
185         };
186         // Делаем паузу между действиями
187         thread::sleep(Duration::from_millis(rng.gen_range
(10..40)));
188
189         // Берем вторую вилку
190         send_state(second_state, &tx);
191         let second_guard = loop {
192             if !running.load(Ordering::Relaxed) {
193                 // Освобождаем первую вилку и завершаемся
194                 drop(first_guard);
195                 waiter.release();
196                 return;
197             }
198             if let Ok(g) = forks[second_idx].try_lock() {
199                 break g;
200             }
201             thread::sleep(Duration::from_millis(5));
202         };

```



```

203         thread :: sleep (Duration :: from_millis (rng.gen_range
(10..40)));
204
205         // EAT
206         send_state (State :: Eating, &tx);
207         thread :: sleep (Duration :: from_millis (rng.gen_range
(80..220)));
208
209         // PUT
210         send_state (State :: PuttingDown, &tx);
211         drop (second_guard);
212         drop (first_guard);
213         waiter.release();
214
215         thread :: sleep (Duration :: from_millis (rng.gen_range
(10..40)));
216     }
217 });
218
219     thread_handles.push (thread_handle);
220 }
221
222 // Считаем время
223 thread :: sleep (Duration :: from_secs (SECONDS));
224
225 // Завершаем работу философов
226 running.store (false, Ordering :: Relaxed);
227
228 // Закрываем канал логгера
229 drop (tx);
230
231 // Ждем завершения всех потоков
232 for h in thread_handles {
233     let _ = h.join();
234 }
235
236 // Ждем завершения логгера
237 let _ = logger.join();
238
239 println!("Simulation finished cleanly");
240 }

```

Вывод программы

Листинг 2: Вывод программы

1	t (ms)		P00	P01	P02	P03	P04
2			-----				
3	0		THINK	THINK	THINK	THINK	THINK
4	0		THINK	THINK	THINK	THINK	THINK
5	0		THINK	THINK	THINK	THINK	THINK
6	0		THINK	THINK	THINK	THINK	THINK
7	0		THINK	THINK	THINK	THINK	THINK
8	111		THINK	THINK	TAKE_L	THINK	THINK
9	133		THINK	THINK	TAKE_R	THINK	THINK
10	144		THINK	THINK	EAT	THINK	THINK
11	145		THINK	THINK	EAT	THINK	TAKE_R
12	164		THINK	THINK	EAT	THINK	TAKE_L
13	187		THINK	THINK	EAT	THINK	EAT
14	187		THINK	TAKE_L	EAT	THINK	EAT
15	201		THINK	TAKE_R	EAT	THINK	EAT
16	233		TAKE_L	TAKE_R	EAT	THINK	EAT
17	249		TAKE_L	TAKE_R	EAT	TAKE_L	EAT
18	265		TAKE_L	TAKE_R	PUT	TAKE_L	EAT
19	278		TAKE_L	EAT	PUT	TAKE_L	EAT
20	284		TAKE_L	EAT	PUT	TAKE_R	EAT
21	295		TAKE_L	EAT	THINK	TAKE_R	EAT
22	384		TAKE_L	EAT	THINK	TAKE_R	PUT
23	406		TAKE_L	EAT	THINK	TAKE_R	THINK
24	410		TAKE_R	EAT	THINK	TAKE_R	THINK
25	420		TAKE_R	EAT	THINK	EAT	THINK
26	437		TAKE_R	PUT	THINK	EAT	THINK
27	457		EAT	PUT	THINK	EAT	THINK
28	462		EAT	THINK	THINK	EAT	THINK
29	489		EAT	THINK	THINK	EAT	TAKE_R
30	494		EAT	THINK	TAKE_L	EAT	TAKE_R
31	512		EAT	THINK	TAKE_R	EAT	TAKE_R
32	545		PUT	THINK	TAKE_R	EAT	TAKE_R
33	558		PUT	THINK	TAKE_R	EAT	TAKE_L
34	563		PUT	TAKE_L	TAKE_R	EAT	TAKE_L
35	574		THINK	TAKE_L	TAKE_R	EAT	TAKE_L
36	584		THINK	TAKE_R	TAKE_R	EAT	TAKE_L
37	595		THINK	TAKE_R	TAKE_R	PUT	TAKE_L
38	618		THINK	TAKE_R	TAKE_R	PUT	EAT
39	629		THINK	TAKE_R	EAT	PUT	EAT
40	630		THINK	TAKE_R	EAT	THINK	EAT
41	700		TAKE_L	TAKE_R	EAT	THINK	EAT
42	732		TAKE_L	TAKE_R	EAT	THINK	PUT
43	742		TAKE_L	TAKE_R	PUT	THINK	PUT
44	742		TAKE_R	TAKE_R	PUT	THINK	PUT
45	750		TAKE_R	TAKE_R	PUT	THINK	THINK

```

46 762 | TAKE_R TAKE_R THINK THINK THINK
47 772 | TAKE_R EAT THINK THINK THINK
48 842 | TAKE_R EAT TAKE_L THINK THINK
49 856 | TAKE_R EAT TAKE_L TAKE_L THINK
50 868 | TAKE_R EAT TAKE_L TAKE_L TAKE_R
51 889 | TAKE_R EAT TAKE_L TAKE_R TAKE_R
52 911 | TAKE_R EAT TAKE_L EAT TAKE_R
53 984 | TAKE_R PUT TAKE_L EAT TAKE_R
54 997 | EAT PUT TAKE_L EAT TAKE_R
55 999 | EAT THINK TAKE_L EAT TAKE_R
56 1012 | EAT THINK TAKE_L PUT TAKE_R
57 1015 | EAT THINK TAKE_R PUT TAKE_R
58 1124 | PUT THINK TAKE_R PUT TAKE_R
59 1231 | PUT TAKE_L TAKE_R PUT TAKE_R
60 Simulation finished cleanly

```

Вывод

В данной лабораторной работе была реализована модель задачи о пяти обедающих философах с использованием потоков и механизмов синхронизации в языке Rust. Для предотвращения взаимоблокировок была применена стратегия официанта, которая ограничивает количество философов, одновременно пытающихся взять вилки.

Характеристики компьютера

Листинг 3: Характеристики компьютера

```

1 > inxi --cpu --memory
2 Memory:
3   System RAM: total: 16 GiB available: 14.8 GiB used: 10.48 GiB (70.8%)
4   Array-1: capacity: 64 GiB slots: 4 modules: 4 EC: None
5   Device-1: Channel-A DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
6   Device-2: Channel-B DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
7   Device-3: Channel-C DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
8   Device-4: Channel-D DIMM 0 type: LPDDR5 size: 4 GiB speed: 6400 MT/s
9 CPU:
10  Info: 6-core model: AMD Ryzen 5 7640HS w/ Radeon 760M Graphics bits:
    64
11   type: MT MCP cache: L2: 6 MiB

```

12	Speed (MHz): avg: 1200 min/max: 400/5017 cores: 1: 1390 2: 1100 3:
13	1100 4: 1100 5: 1100 6: 1100 7: 1396 8: 1100 9: 1432 10: 1100 11: 1389 12: 1100